GenAI Lab: Experiment Guide and Viva Questions

For Krishika Jalali (2023800040). Part A covers your 9 lab experiments. Part B covers the 16 offline ESE problem statements (the code files are in ESE_codes.zip). Each experiment has: what happens, how it works, and viva questions with short answers.

Part A: Your Lab Experiments

Lab 1: Autoencoder and VAE on MNIST

What happens. Part 1 builds a plain autoencoder (784 → 256 → 64 → 256 → 784), trains it with MSE, and shows originals vs reconstructions, latent vectors, and an interpolation between two digits (3 → 8). Part 2 builds a VAE (latent dim 20) that learns a distribution instead of a point, and generates brand-new digits by sampling random noise.

How. The VAE encoder outputs mu and logvar. We sample z = mu + sigma * eps (eps ~ N(0,1)); this is the reparameterization trick, so gradients can flow through the sampling. Loss = BCE reconstruction + KL divergence to N(0,I). After training, throw the encoder away: sample z ~ N(0,I) and pass it through the decoder.

Viva.

- **Q: Why can't a plain autoencoder generate new images?** A: Its latent space has holes and no known distribution to sample from, so random points decode to garbage.
- **Q: What is the reparameterization trick?** A: Write z = mu + sigma * eps so the randomness sits in eps, and mu and sigma stay differentiable.
- **Q: Write the VAE loss.** A: Reconstruction loss + KL(q(z|x) || N(0,I)). KL has the closed form -0.5 * sum(1 + logvar - mu² - exp(logvar)).
- **Q: Why predict logvar, not variance?** A: The network can output any real number and exp(logvar) is always positive. It is also numerically stable.
- **Q: What happens if the KL term is dropped? If the reconstruction term is dropped?** A: Without KL it becomes a plain AE with a gappy latent space. Without reconstruction every input collapses to N(0,I) and the latent code is useless (posterior collapse).
- **Q: Why are VAE images blurry?** A: Pixel-wise losses average over possibilities, and the KL term trades sharpness for a smooth latent space.

- **Q: What is ELBO?** A: The evidence lower bound on log-likelihood. Maximising ELBO equals minimising the VAE loss.
- **Q: Why does your KL loss go up early in training?** A: The encoder first uses the latent space freely to reconstruct, then the KL pull stabilises it.
- **Q: Name a VAE variant.** A: Beta-VAE (weights KL for disentangling), Conditional VAE (generate a chosen digit), ConvVAE (sharper images).

Lab 2: GAN on MNIST

What happens. A Generator (noise 100 → 256 → 512 → 1024 → 784, Tanh) and a Discriminator (784 → 1024 → 512 → 256 → 1, Sigmoid) play a game. 50 epochs, Adam lr 0.0002, betas (0.5, 0.999). You plotted G/D loss, image grids at epochs 1 to 50, and a proxy FID (92.67) and proxy IS (7.48).

How. Each batch: (a) train D on real images (label 1) and fake images (label 0); (b) train G so D says "real" for fakes (label 1). Images are normalised to [-1,1] to match Tanh. Loss is BCE.

Viva.

- **Q: Write the GAN objective.** A: min_G max_D E[log D(x)] + E[log(1 - D(G(z)))].
- **Q: What is the non-saturating generator loss?** A: G minimises -log D(G(z)) instead of log(1 - D(G(z))), which gives stronger gradients early.
- **Q: What is mode collapse?** A: G produces only a few kinds of outputs (e.g., only a few digits) because they fool D.
- **Q: What is D loss at equilibrium?** A: About log 4 = 1.386 (total over real and fake). With the averaged loss in your code, about 0.693.
- **Q: Why do GAN losses oscillate?** A: Two networks chase a moving target, so a falling loss does not mean better images. Look at samples.
- **Q: Why LeakyReLU, Dropout in D, Tanh in G, beta1 = 0.5?** A: LeakyReLU avoids dead neurons, Dropout slows D, Tanh matches [-1,1] pixels, beta1 = 0.5 stabilises adversarial training.
- **Q: What are IS and FID?** A: IS rewards confident and diverse classifications (higher is better). FID compares Gaussian fits of real vs fake features (lower is better). FID is more reliable.
- **Q: Why a proxy FID for MNIST?** A: Real FID uses ImageNet Inception features, which do not suit 28x28 grayscale digits.

- **Q: How to fix instability?** A: Label smoothing, WGAN-GP, DCGAN, lower D capacity, noise in D inputs.
- **Q: Applications?** A: Medical image synthesis, data augmentation, defect detection.

Lab 3: Fine-tuning GPT-2 / DistilGPT2

What happens. A pretrained DistilGPT2 is fine-tuned on a small AI-topic text corpus with causal language modelling. You then generate text from prompts and study temperature, top-k, top-p, and plot loss/perplexity vs epochs.

How. Tokenization turns text into token IDs. Embedding turns IDs into vectors. The model predicts the next token; loss is cross-entropy; perplexity = exp(loss).

Viva.

- **Q: Tokenization vs embedding?** A: Tokenization maps text to integer IDs. Embedding maps those IDs to dense vectors the model can learn.
- **Q: What is causal language modelling?** A: Predict the next token using only previous tokens (masked attention).
- **Q: What is transfer learning / fine-tuning?** A: Start from pretrained weights and train a little on your own data so it picks up your domain style.
- **Q: Temperature, top-k, top-p?** A: Temperature rescales logits (low = safe, high = random). Top-k keeps the k best tokens. Top-p keeps the smallest set whose probability adds to p.
- **Q: What is perplexity?** A: exp(average cross-entropy). It is the model's effective number of choices per token. Lower is better.
- **Q: Why set pad_token = eos_token?** A: GPT-2 has no pad token by default.
- **Q: Why can fine-tuning on a tiny corpus overfit?** A: The model memorises. Perplexity on training data drops but quality does not generalise.

Lab 4: Text-to-Image with DALL·E Mini

What happens. A text prompt is turned into an image by a pretrained model (DALL·E Mini / Craiyon style). You vary the prompt and compare results.

How. The text goes through a seq2seq transformer that predicts discrete image tokens. A VQGAN decoder turns the tokens into pixels. Several candidates are generated and ranked by CLIP.

Viva.

- **Q: What does a text-to-image model need?** A: A text encoder and an image generator conditioned on the text.
- **Q: What is VQGAN / discrete image tokens?** A: An autoencoder that compresses an image into a sequence of codebook indices, so images can be generated like text.
- **Q: What is CLIP used for here?** A: It scores how well each generated image matches the prompt, to pick the best.
- **Q: Why is DALL·E Mini lower quality than Stable Diffusion?** A: It is smaller and uses token generation rather than latent diffusion.
- **Q: How does the prompt affect output?** A: Specific content and style words steer subject, texture and colour.

Lab 5: Autoencoder-based Face Swapping (educational)

What happens. You trained a shared encoder and two decoders (A and B) on 64x64 Olivetti faces using NumPy only, with augmentation. Swap = encode A, decode with B. Reported reconstruction MSE: A 0.00315, B 0.00513.

How. The shared encoder keeps pose and expression (identity-neutral). Each decoder learns one person′s appearance. Crossing the wires gives B′s face with A′s pose. Manual backprop with Adam, tanh latent, sigmoid output.

Viva.

- **Q: Why one encoder and two decoders?** A: The shared encoder cannot store person-specific detail, so it keeps general structure; decoders store identity.
- **Q: Why augmentation?** A: Only 10 photos per person; shifts, flips and brightness make more training data without changing identity.
- **Q: Why are swaps blurry?** A: Small data, a simple MLP, and MSE loss.
- **Q: Ethical concerns of deepfakes?** A: Misinformation, consent, fraud. Mention detection and watermarking.
- **Q: How do real deepfake systems improve it?** A: Convolutional encoders, GANs, face alignment and blending.

Lab 6: AI Music with OpenAI Jukebox

What happens. Jukebox (1b_lyrics) generated 18 s of Jazz (genre = Jazz, artist unknown, no lyrics) in about 8.6 minutes, saved as WAV. Integrity checks passed. Waveform and spectrogram were plotted; a second genre was compared.

How. A 3-level VQ-VAE compresses raw audio to discrete codes. A Transformer prior generates top-level codes from the genre/artist/lyrics labels; upsamplers add detail; the decoder makes audio. You decoded from top-level codes only, so it sounds rough.

Viva.

- **Q: Why is raw audio hard to generate?** A: 44,100 samples per second means huge sequences.
- **Q: What is a VQ-VAE?** A: A VAE with a discrete codebook latent space.
- **Q: Why three levels?** A: Top level captures long-range structure, bottom level captures fine timbre.
- **Q: Conditioning in Jukebox?** A: Genre, artist and lyrics only. No direct mood or tempo control. Mood is approximated by choosing the genre.
- **Q: Waveform vs spectrogram?** A: Waveform is amplitude vs time. Spectrogram is frequency content vs time (STFT).
- **Q: How do you know the WAV is valid?** A: It loads, duration matches, max amplitude above zero, no NaN/Inf.
- **Q: Why was it slow?** A: Autoregressive sampling over thousands of tokens in windows.
- **Q: Newer alternatives?** A: MusicGen, MusicLM, Suno.

Lab 7: CycleGAN (Horse to Zebra)

What happens. Two generators (G_AB, G_BA) and two 70x70 PatchGAN discriminators were trained on unpaired horse/zebra images (256x256, batch 4, 10 epochs, lr 2e-4, betas (0.5, 0.999), lambda_cycle = 10, identity 5). Final losses: cycle 0.2694, identity 0.2621.

How. Total loss = adversarial (LSGAN/MSE) + 10 * cycle loss (L1 of F(G(x)) vs x) + identity loss. Image pool of 50, linear LR decay, AMP.

Viva.

- **Q: Why does it work without paired images?** A: Cycle consistency forces the translation to keep the content so it can be reversed.

- **Q: Write the loss.** A: L = L_GAN(G,D_Y) + L_GAN(F,D_X) + lambda * (|F(G(x)) - x| + |G(F(y)) - y|).
- **Q: What is the identity loss?** A: G(y) should stay close to y so colours are not changed needlessly.
- **Q: What is PatchGAN?** A: A discriminator that judges each 70x70 patch as real or fake, so it focuses on local texture.
- **Q: Why LSGAN (MSE)?** A: More stable gradients than BCE.
- **Q: What is the image history buffer?** A: D is also shown older fake images, which reduces oscillation.
- **Q: Why InstanceNorm and reflection padding?** A: Better style-transfer quality and fewer border artifacts.
- **Q: Limits?** A: Cannot do large shape changes, leaves artifacts. Related: AlignFlow (exactly invertible), StarGAN (many domains).

Lab 8: Chatbot with DistilGPT2

What happens. An interactive loop: tokenize input, generate with max_new_tokens = 50, temperature 0.7, top-k 50, top-p 0.95, repetition penalty 1.1, decode, print, repeat until `exit`. You tested 12 query types and a multi-turn version with history.

How. DistilGPT2 has 6 blocks and about 82M parameters (distilled from GPT-2 small). It only continues text, so answers are often off-topic.

Viva.

- **Q: What is knowledge distillation?** A: A small student learns to match the teacher's output probabilities.
- **Q: Why are the answers poor?** A: It is a base LM, not instruction-tuned. It predicts likely text and does not answer questions.
- **Q: What does repetition penalty do?** A: It lowers the logit of tokens already generated, to stop loops.
- **Q: What does do_sample=True do?** A: Enables temperature/top-k/top-p sampling instead of greedy decoding.
- **Q: How to give memory across turns?** A: Add the conversation history to the prompt. It is limited by the context window and the model still may ignore it.
- **Q: What makes ChatGPT-style bots better?** A: Instruction tuning and RLHF, much larger scale, and role-formatted chat data.

- **Q: What is hallucination?** A: Fluent but false output, since the model predicts plausible text.

Lab 9: Diffusion (DDPM) and Stable Diffusion

What happens. Part 1: DDPM on two moons. Part 2: compared linear vs cosine schedules for T in {10, 50, 200, 1000}. Part 3: Stable Diffusion v1.5 with prompt, guidance scale and steps varied.

How. Forward: x_t = sqrt(alpha_bar_t) * x0 + sqrt(1 - alpha_bar_t) * eps. The network predicts eps; loss is MSE. Reverse sampling starts from pure noise. Stable Diffusion does this in the VAE latent space with a text-conditioned U-Net. Classifier-free guidance: eps = eps_uncond + w * (eps_cond - eps_uncond).

Viva.

- **Q: What does the network predict?** A: The noise that was added.
- **Q: Why does x_t have a closed form?** A: Gaussian noise steps combine, so any step can be sampled directly with alpha_bar_t.
- **Q: Linear vs cosine?** A: Cosine destroys signal more gradually. Linear T=10 failed in your run because the betas hit the 0.999 clamp.
- **Q: Why should signal at T be near zero?** A: x_T must be pure noise so sampling can start from N(0,I).
- **Q: Guidance scale effect?** A: Low = muddy and ignores the prompt. 7 to 12 = good. Very high = oversaturated.
- **Q: Why fewer steps are faster but worse?** A: Fewer denoising iterations means a rougher image (2 steps blurry, 10 to 20 clean).
- **Q: Why is Stable Diffusion cheaper?** A: Diffusion in a small latent space, not pixels.
- **Q: Diffusion vs GAN?** A: Diffusion trains stably and covers modes but is slow at sampling. GANs are fast but unstable.
- **Q: Is lower loss better samples?** A: Not necessarily. Your linear loss was lower but the cosine schedule has a different noise-level mix.

Part B: ESE Offline Problem Statements (PS1 to PS16)

These run with no internet and no datasets. See ESE_codes.zip and its README for run steps.

PS1 and PS2: Markov chain text generation

What happens. A table maps each word (or k characters) to the list of things that followed it in a short typed text. Generation repeatedly picks a random next item.

Viva.

- **Q: What is a Markov chain?** A: The next state depends only on the current state.
- **Q: How is this like an LLM?** A: Both do next-token prediction. An LLM uses a neural network and long context; this uses a lookup table.
- **Q: Why does repeated text appear?** A: The corpus is tiny, so few next options exist.
- **Q: Effect of order k?** A: Higher k copies the source more closely; lower k is more random.
- **Q: Why does picking from a list with duplicates work?** A: Frequent followers appear more often, so the choice follows the real probabilities.

PS3: Bigram model, sampling controls, perplexity

What happens. Count word pairs, turn counts into probabilities, then sample with temperature, top-k and top-p. Compute perplexity.

Viva.

- **Q: Effect of temperature?** A: Low (0.3) repeats safe words; high (2.0) produces nonsense.
- **Q: Difference between top-k and top-p?** A: Top-k has a fixed count; top-p adapts to the model's confidence.
- **Q: Why add smoothing?** A: So unseen pairs are not zero probability (log of zero).
- **Q: Formula for perplexity?** A: exp(-average log probability of the true next word).

PS4: Chatbot loop

What happens. Reads input in a while loop, checks keywords, otherwise generates a Markov reply seeded with the user's last known word, stops at `exit`.

Viva.

- **Q: How does the loop end?** A: When the user types exit.
- **Q: Is it context-aware?** A: No, each turn is independent; history would need to be added.
- **Q: How is it different from DistilGPT2?** A: Same generate-and-decode idea, but a table replaces the Transformer.

PS5, PS6, PS7: Genetic and evolutionary generation

What happens. Start from random pixels (or letters). Fitness counts how many match the target (cross, or a sentence). The best individuals are kept, mixed (crossover) and flipped randomly (mutation) until a perfect match. PS6 is a simpler (1+1) evolution: flip one pixel, keep if not worse.

Viva.

- **Q: What is the fitness function?** A: Number of pixels equal to the target.
- **Q: Roles of selection, crossover, mutation?** A: Selection keeps good candidates, crossover combines them, mutation adds variety to avoid getting stuck.
- **Q: What is elitism?** A: Always copy the best individual into the next generation.
- **Q: Why is this "generative"?** A: It creates new images from noise through iterative optimisation, with no training data.
- **Q: What if the mutation rate is too high or too low?** A: Too high is random search; too low gets stuck in local optima.
- **Q: How is it different from a neural generator?** A: It optimises one image against a known target, and does not learn a distribution.

PS8: Autoencoder from scratch

What happens. Synthetic 8x8 shapes (bar or square) are compressed to 4 numbers and rebuilt. Shows reconstructions, a latent vector, interpolation between a horizontal and vertical bar, and a loss curve.

Viva.

- **Q: Why a bottleneck?** A: It forces a compact representation instead of copying.
- **Q: Why does interpolation look like a morph?** A: Decoding points between two latent codes gives in-between shapes.
- **Q: What does backprop do here?** A: Applies the chain rule layer by layer from loss to weights; Adam updates them.

PS9: VAE from scratch

What happens. Same data with a 2-D latent. Loss is reconstruction + KL. New images are made by decoding random normal samples. A scatter plot shows the latent space coloured by class.

Viva.

- **Q: Gradient of KL?** A: dKL/dmu = mu; dKL/dlogvar = 0.5 * (exp(logvar) - 1).
- **Q: Why a 2-D latent?** A: So the latent space can be plotted.
- **Q: Does the latent space cluster by class?** A: Roughly yes, even though labels were not used.

PS10: Basic GAN (1-D)

What happens. G maps noise to numbers, D tells real (N(4,1.25)) from fake. Plot loss curves and compare histograms. In this run it learns the mean well but the spread only partly.

Viva.

- **Q: Why is the generated spread too small?** A: Partial mode collapse; G favours the region D finds most convincing.
- **Q: Why detach / not update G while training D?** A: D's update must not change G, so the two take turns.
- **Q: D loss near 1.386 means?** A: D is at chance; it cannot tell real from fake.
- **Q: How to improve it?** A: Lower learning rate, more steps, minibatch discrimination, WGAN.

PS11: Diffusion on two moons

What happens. Train a small network to predict noise for noisy two-moons points at random time steps. Sample by starting from random noise and denoising 100 steps. Compare linear and cosine schedules by distance to real data.

Viva.

- **Q: Forward vs reverse process?** A: Forward adds noise on a schedule; reverse is learned and removes it.
- **Q: What is alpha_bar?** A: Cumulative product of (1 - beta); it sets the signal level.
- **Q: Why add noise during sampling?** A: DDPM reverse steps are stochastic. The last step has none.
- **Q: Why clip the predicted x0?** A: Keeps sampling stable when the last cosine beta is near 1.

PS12: Face swap (toy)

What happens. Synthetic faces: A is a circle, B is a square, both with a bright blob for pose. A shared 3-number encoder and two decoders are trained. Decoding A's code with B's decoder draws a square face with A's blob position (error about 1 pixel).

Viva.

- **Q: Why does a small latent help the swap?** A: It forces the shared code to keep pose, not identity.
- **Q: What is swapped?** A: Which decoder receives the code.
- **Q: Why can a big latent fail?** A: The code can carry identity, so the other decoder sees an unfamiliar input.

PS13: Toy CycleGAN

What happens. Two 1-D domains (A: N(0,1), B: N(4,2)), linear G and F, logistic discriminators, loss = adversarial + 10 * L1 cycle. After training G(A) matches B and F(B) matches A, and the cycle error is near zero.

Viva.

- **Q: Why two generators and two discriminators?** A: One pair per direction (A to B and B to A).

- **Q: Does cycle loss alone force the correct target distribution?** A: No, any invertible pair satisfies it. The adversarial loss supplies the target domain.
- **Q: Why lambda = 10?** A: Gives cycle consistency strong weight compared with the adversarial term.

PS14: Music with Markov chain, WAV, spectrogram

What happens. Notes from a pentatonic scale are chosen by a transition matrix (jazz: smooth steps, long notes; rock: repeats the root, short notes, harsher tone). Audio is synthesised, saved as WAV, reloaded, checked, and shown as waveform and spectrogram.

Viva.

- **Q: What does a spectrogram show?** A: Frequency content over time; horizontal lines are held notes.
- **Q: How is it computed?** A: Short windows, FFT of each, magnitude in dB.
- **Q: Sample rate meaning?** A: Number of amplitude samples per second (here 22,050).
- **Q: Difference from Jukebox?** A: This models symbolic notes with a Markov chain; Jukebox generates raw audio with VQ-VAE and Transformers.

PS15: Tokenization, embeddings, perplexity

What happens. Words become IDs, IDs become trainable vectors, a softmax layer predicts the next word. Loss and perplexity are plotted per epoch. Perplexity falls to about 1.3 because the corpus is tiny.

Viva.

- **Q: Tokenization vs embedding?** A: IDs vs dense learned vectors.
- **Q: Why does perplexity drop to near 1?** A: The model memorises a tiny corpus.
- **Q: Which part is the language model?** A: Embedding table + output layer, predicting the next token.

PS16: Self-attention

What happens. Queries, keys and values are computed from embeddings. Scores = QK^T / sqrt(d), a causal mask hides future words, softmax gives weights, and the output is a weighted sum of values.

Viva.

- **Q: Why divide by sqrt(d)?** A: Keeps the scores from growing large and saturating the softmax.
- **Q: Why a causal mask?** A: GPT-style models must not see future tokens when predicting the next.
- **Q: Query, key and value?** A: Query asks, key says what a token offers, value is the content that gets mixed.
- **Q: Why is attention better than an RNN?** A: Parallel computation and direct long-range links.

Quick Reference

- **Autoencoder:** compress and rebuild. **VAE:** AE plus a distribution and KL, so it can generate.
- **GAN:** G vs D, adversarial. **CycleGAN:** two GANs plus cycle loss for unpaired translation.
- **Diffusion:** add noise step by step, learn to predict it, reverse from pure noise.
- **GPT / chatbot:** next-token prediction, sampling by temperature, top-k, top-p.
- **Metrics:** Perplexity (language, lower better); IS (higher better); FID (lower better).
- **Offline ESE tip:** know the screenshots' three ideas well: Markov text (random only), genetic image evolution (numpy + matplotlib), evolutionary 3x3 image (numpy).